

mastermind_v2

Variation de la difficulté du jeu

À partir de la première version du Mastermind (mastermind_v1.cpp), modifier le programme afin de rendre la partie configurable et d'améliorer la saisie des réponses. Au début de chaque partie, le joueur doit pouvoir choisir le nombre de chances entre 6 et 12, la longueur de l'énigme entre 3 et 12, ainsi que la difficulté entre 2 et 8. La difficulté correspond au nombre de lettres disponibles parmi : `q w e r t y u i`. Créer une fonction responsable de cette initialisation.

Indice

Une fonction ne peut retourner directement qu'une seule valeur avec `return`. Le passage par référence permet toutefois à une fonction de modifier plusieurs variables reçues en paramètres. Une fonction d'initialisation pourrait donc recevoir par référence les variables : `chances`, `longueur`, `difficulte`. Créer également une fonction permettant de saisir un entier compris entre un minimum et un maximum. Elle devra gérer les valeurs hors limites ainsi qu'une saisie invalide avec `cin`.

Éliminer les variables globales modifiables

Les tableaux `enigme`, `reponseJoueur` et `copieEnigme` ne doivent plus être globaux. Ils doivent être déclarés dans `main()` et transmis aux fonctions qui en ont besoin. Les constantes comme `TAILLE_MAX` et le tableau contenant les lettres valides peuvent demeurer globales.

Indice

Un tableau transmis à une fonction n'est pas copié au complet. La fonction travaille avec le tableau original. On peut donc considérer, pour cet exercice, qu'un tableau est lui aussi passé par référence. Il faudra par conséquent modifier plusieurs fonctions existantes afin de leur transmettre les tableaux, la longueur ou la difficulté dont elles ont maintenant besoin. Par exemple, `genererEnigme()` devra désormais connaître la longueur, la difficulté et le tableau dans lequel placer l'énigme.

Améliorer la saisie d'une réponse

Dans la première version, `saisirEtAfficherLettre()` lit une lettre, l'affiche immédiatement et la retourne. Cette façon de faire convient lorsque la réponse est saisie strictement de gauche à droite, mais elle devient insuffisante puisque le joueur doit maintenant pouvoir revenir sur sa réponse. Cette fonction pourra donc être abandonnée ou remplacée.

Lors d'une tentative, le joueur doit pouvoir utiliser :

- Flèche gauche : déplacer le curseur vers la gauche
- Flèche droite : déplacer le curseur vers la droite
- RETOUR : confirmer la réponse
- Lettre valide : écrire ou remplacer la lettre courante

Créer un `enum` permettant de représenter `GAUCHE`, `DROITE` et `RETOUR`. Une nouvelle fonction pourrait être responsable de lire une touche et une autre de vérifier si cette touche est permise.

Indice

Les nouvelles fonctions pourraient avoir des responsabilités semblables à : `saisirTouche(...)`
`toucheEstValide(...)`

La fonction `lettreEstValide()` demeure utile, mais elle devra maintenant tenir compte de la difficulté choisie.

Avec `_getch()`, une touche spéciale peut nécessiter deux lectures. Le premier appel peut retourner `0` ou `224`; dans ce cas, un second `_getch()` permet d'obtenir le code de la touche. Un `unsigned char` peut être utilisé temporairement pour pouvoir représenter correctement la valeur `224`.

Permettre le déplacement dans la réponse

La fonction `recupererReponse()` doit être modifiée de façon importante. Elle ne doit plus simplement effectuer une boucle un nombre fixe de fois. Elle doit plutôt continuer tant que le joueur n'a pas appuyé sur `RETOUR`. Il faudra conserver l'indice de la position actuellement modifiée dans `reponseJoueur`. La flèche gauche diminue cet indice, la flèche droite l'augmente, sans jamais dépasser les limites du tableau utilisé. Lorsqu'une lettre valide est saisie, elle est placée à la position actuelle et remplace la lettre qui s'y trouvait déjà. Les fonctions suivantes de `cvm_25.h` permettront de déplacer le curseur dans la console :

```
wherex()  
wherey()  
gotoxy()
```

Il peut également être utile de créer une fonction qui vide ou initialise `reponseJoueur` avant chaque nouvelle tentative.

Adapter les fonctions existantes

La logique de vérification du Mastermind demeure la même. Un `X` représente une lettre bien placée et un `.` une lettre présente mais mal placée. Une même lettre de l'énigme ne peut toujours produire qu'un seul indice. Les fonctions existantes devront cependant recevoir explicitement les données dont elles ont besoin plutôt que d'utiliser des variables globales. À la fin de l'exercice, le programme devra :

- Permettre d'initialiser chaque nouvelle partie avec des paramètres différents
- Ne plus utiliser de tableaux globaux modifiables
- Permettre au joueur de corriger sa réponse avant de la confirmer avec `RETOUR`.